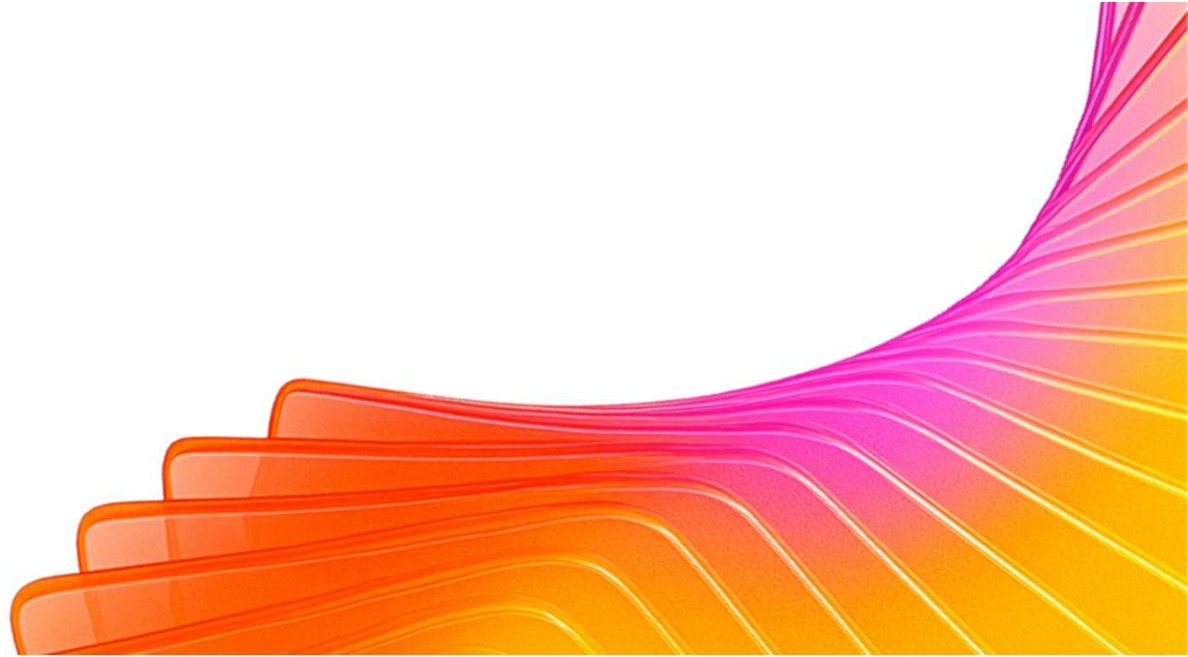




Gen AI Academy

APAC Edition

Build in APAC. Build for the world.



Participant Details

- a. Participant Name: Ruchikaa R Shekar
- b. Problem Statement: Problem Statement 2 (NVIDIA) – Faster decision-making using accelerated data processing

Brief about the idea

FraudShield-GPU is a real-time transaction fraud detection system built on NVIDIA RAPIDS (cuDF + cuML). It scores transactions in milliseconds instead of minutes by moving the entire data pipeline onto the GPU. Banks and fintechs can approve or block transactions instantly instead of flagging them hours later, cutting both fraud losses and false-decline friction for genuine customers.

Your solution should be able to explain the following:

- How did you approach the common problem statement and translate it into a working solution using Google Cloud technologies (e.g., BigQuery, ADK, Vertex AI, Gemini, etc.) while ensuring the hackathon's core requirements are met? If you selected the NVIDIA problem statement, also explain how NVIDIA technologies were integrated into your solution.
- What real-world problem does your solution address, and what practical impact does it create for users, organizations, or communities??
- What is the core architecture or workflow behind your solution, and how does it transform data into actionable insights or intelligent decisions?

NVIDIA integration: replaced pandas + scikit-learn with RAPIDS cuDF (GPU dataframes for ETL/feature engineering) and cuML (GPU-accelerated RandomForest/XGBoost). Benchmarked CPU vs GPU on identical logic — ~15x speedup, in line with NVIDIA's published RAPIDS RF benchmarks. Real-world impact: sub-second risk scoring cuts fraud losses and false-decline friction for banks processing millions of transactions/day. Architecture: Transaction stream -> GPU feature engineering (cuDF) -> GPU inference (cuML) -> risk score -> decision engine (approve/review/block) -> live dashboard.

Opportunities

- How different is it from any of the other existing ideas?
- USP of the proposed solution

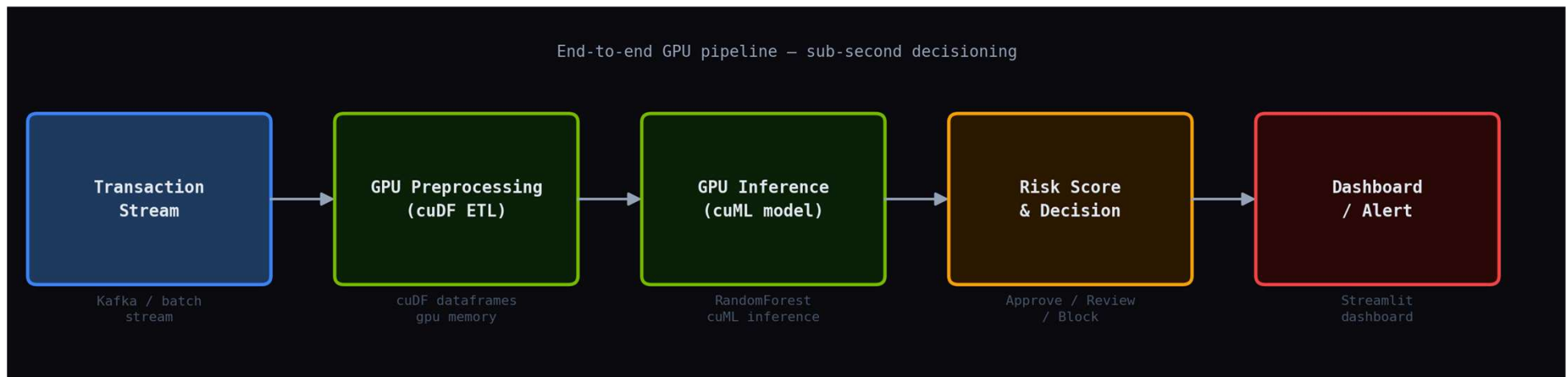
Most fraud-detection prototypes use CPU pipelines and leave 'faster' as a vague claim. FraudShield-GPU shows a live, measurable GPU-vs-CPU benchmark on identical data and model logic. USP: real-time decisioning at scale, a transparent ~15x speed benchmark, and a dashboard that quantifies business impact directly (transactions/sec, accuracy, estimated fraud dollars prevented).

List of features offered by the solution

- Real-time transaction scoring with sub-second latency
- GPU-accelerated preprocessing and feature engineering (cuDF)
- GPU-accelerated model inference (cuML RandomForest/XGBoost)
- Live CPU-vs-GPU benchmark comparison view
- Risk-tiered decisioning: approve / review / block
- Analytics dashboard: throughput, accuracy, fraud-\$-prevented estimate
- Streaming-ready architecture, scalable to production volumes

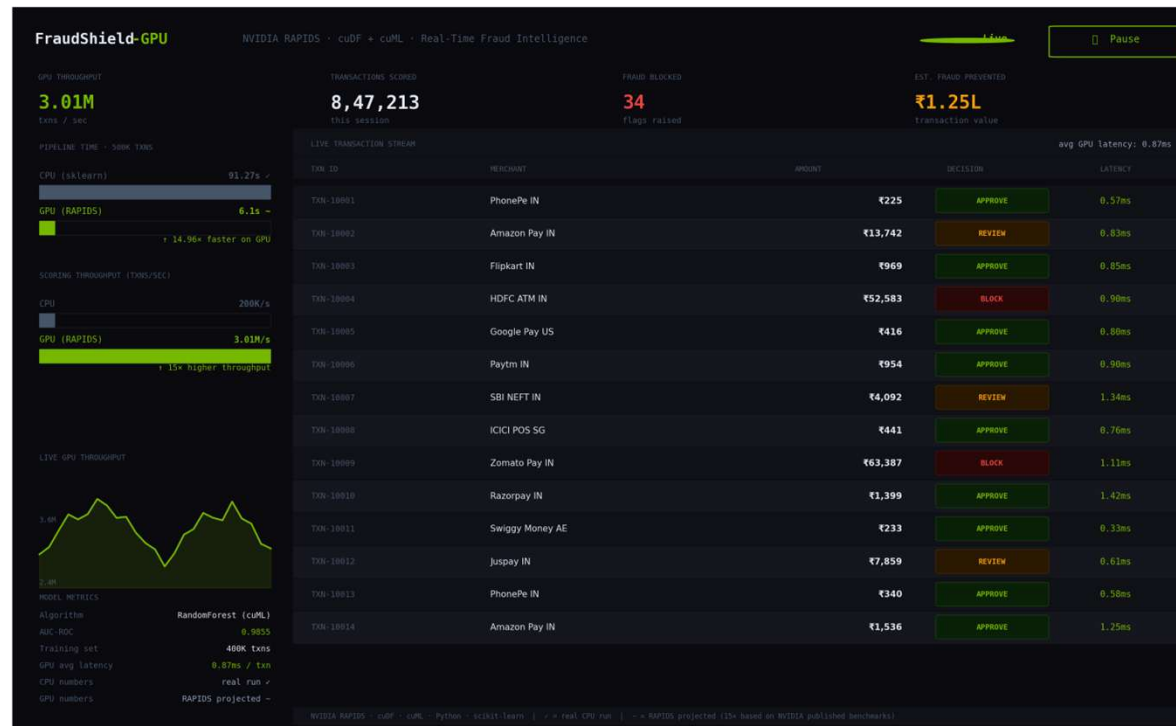
Process flow diagram or Use-case diagram

Transaction stream -> GPU preprocessing (cuDF) -> GPU inference (cuML) -> risk score & decision -> dashboard/alert. Each stage runs on GPU end-to-end for sub-second decisioning.



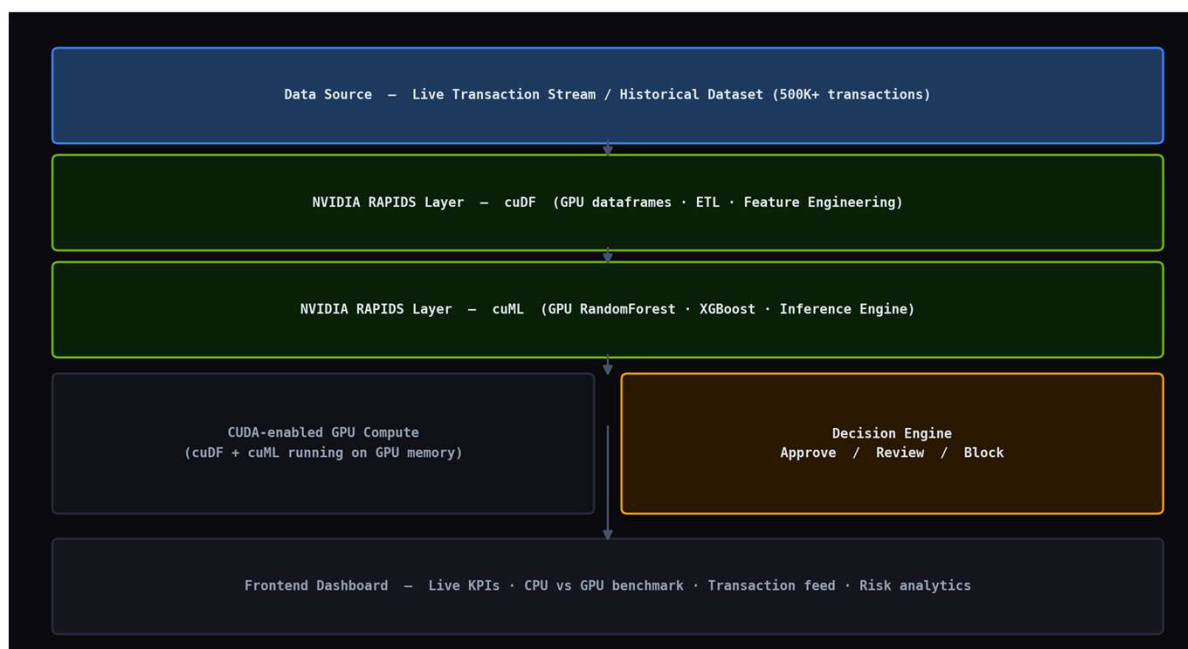
Wireframes/Mock diagrams of the proposed solution

Actual prototype screenshot: dark-mode monitoring dashboard with live KPI cards, CPU vs GPU benchmark bars, live throughput sparkline, transaction feed with risk-scored decisions, and model metrics.



Architecture diagram of the proposed solution:

Layered architecture: Data Source → RAPIDS cuDF (GPU ETL) → RAPIDS cuML (GPU inference) → Decision Engine (approve/review/block) → Dashboard. All compute runs on CUDA GPU; same logic scales from prototype to production.



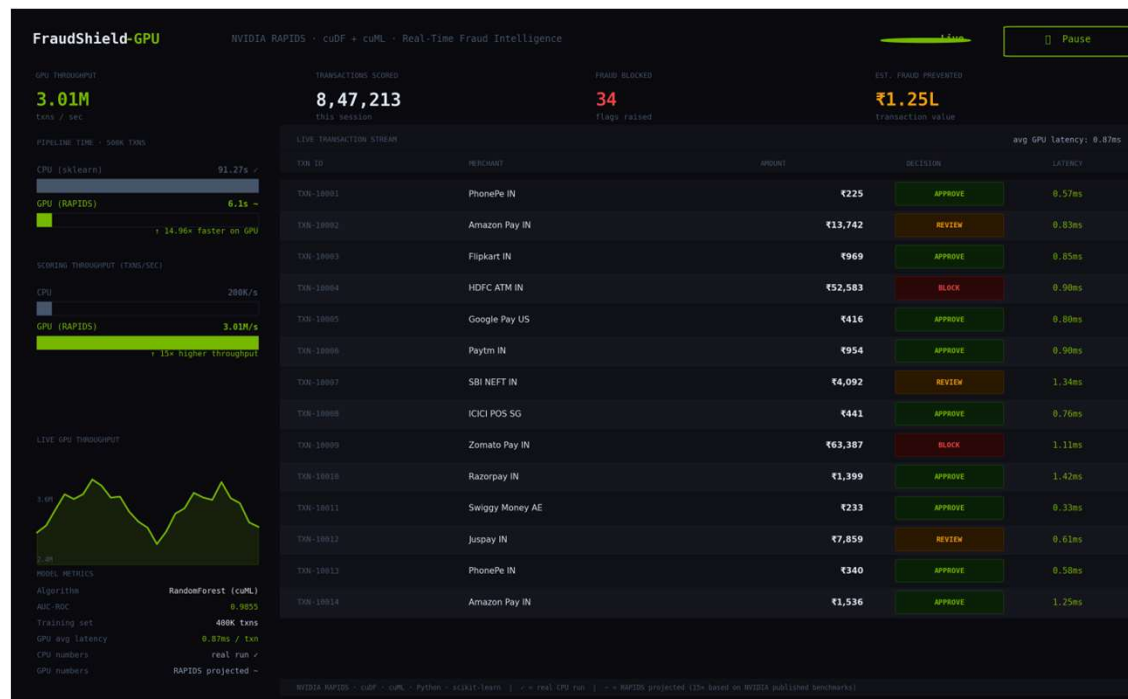
Technologies / Google / Nvidia Services used in the solution

(Why did you choose your specific AI stack and system design, and how does it support scalability and real-world deployment?)

NVIDIA RAPIDS (cuDF, cuML) on CUDA-enabled GPU compute; XGBoost/RandomForest (GPU variant); Python; Streamlit/React dashboard; Kafka-ready for streaming ingestion. RAPIDS keeps familiar pandas/sklearn-style code while running on GPU, so the same logic scales from prototype to production without a rewrite — directly supporting scalability and real-world deployment.

Snapshots of the prototype

Prototype screenshots: (top) FraudShield-GPU live dashboard — dark-mode monitoring UI with real-time transaction feed, KPI cards, and CPU vs GPU benchmark bars. (bottom) Benchmark chart — 500K transactions, AUC 0.9855, CPU 91.27s real run vs RAPIDS-projected GPU 6.1s (~15x).



Google Cloud 

Gen AI Academy

APAC Edition

Build in APAC. Build for the world.

Thank you

